Tyler's Laundry API Documentation

Overview

Tyler's Laundry API is a comprehensive backend service for a laundry management platform built with Node.js, Express.js, and TypeScript. The API provides full CRUD operations for managing users, services, bookings, payments, invoices, receipts, and testimonials.

Base URL: `http://localhost:3000/api/v1/`

Authentication: JWT-based authentication with refresh tokens

Rate Limiting: 100 requests per 15 minutes

Authentication

All API endpoints (except authentication endpoints) require a valid JWT token in the Authorization header.

```
Authorization: Bearer <your-jwt-token>
```

User Roles

- **USER**: Regular customers
- **ADMIN**: System administrators
- **STAFF**: Service staff (can access most admin features)

Health Check

GET /health

Check the health status of the API.

Response (200):

```
{
  "status": "OK",
  "timestamp": "2025-11-08T10:30:00.000Z",
  "uptime": 3600.5,
  "environment": "development",
  "version": "1.0.0"
}
```

Authentication Endpoints

POST /api/v1/auth/signup

Register a new user account.

Request Body:

```
{
  "name": "string (required, min 1 char)",
  "email": "string (required, valid email)",
  "password": "string (required, min 6 chars)"
}
```

Response (201):

```
{
  "message": "User registered successfully. Please check your email for verification code.",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "role": "USER",
    "isActive": false,
    "createdAt": "2025-11-08T10:30:00.000Z"
  }
}
```

POST /api/v1/auth/signin

Authenticate user login.

Request Body:

```
{
  "email": "string (required)",
  "password": "string (required)"
}
```

Response (200):

```
{
  "message": "Login successful",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "role": "USER"
  },
  "tokens": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
    "expiresIn": 3600
  }
}
```

POST /api/v1/auth/refresh-token

Refresh access token using refresh token.

Request Body:

```
{
  "refreshToken": "string (required)"
}
```

Response (200):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "new_refresh_token",
  "expiresIn": 3600
}
```

POST /api/v1/auth/verify-email

Verify user email with verification code.

Request Body:

```
{
  "email": "string (required)",
  "code": "string (required, 6 digits)"
}
```

Response (200):

```
{
  "message": "Email verified successfully"
}
```

POST /api/v1/auth/request-verification-code

Request new email verification code.

Request Body:

```
{
  "email": "string (required)"
}
```

Response (200):

```
{
  "message": "Verification code sent to your email"
}
```

POST /api/v1/auth/request-password-reset

Request password reset token.

Request Body:

```
{
  "email": "string (required)"
}
```

Response (200):

```
{
  "message": "Password reset instructions sent to your email"
}
```

POST /api/v1/auth/validate-reset-token

Validate password reset token.

Request Body:

```
{
  "resetToken": "string (required)"
}
```

Response (200):

```
{
  "message": "Token is valid"
}
```

POST /api/v1/auth/reset-password

Reset password using reset token.

Request Body:

```
{
  "email": "string (required)",
  "newPassword": "string (required, min 6 chars)"
}
```

Response (200):

```
{
  "message": "Password reset successfully"
}
```

POST /api/v1/auth/change-password

Change user password (requires authentication).

Request Body:

```
{
  "oldPassword": "string (required)",
  "newPassword": "string (required, min 6 chars)"
}
```

Response (200):

```
{
  "message": "Password changed successfully"
}
```

POST /api/v1/auth/logout

Logout user (requires authentication).

Response (200):

```
{
  "message": "Logged out successfully"
}
```

DELETE /api/v1/auth/:id

Soft delete user account (requires authentication).

Request Body:

```
{
  "deletionReason": "string (optional)",
  "password": "string (required)"
}
```

Response (200):

```
{
  "message": "Account deleted successfully"
}
```

POST /api/v1/auth/admin/signup

Register admin account.

Request Body:

```
{
  "name": "string (required)",
  "email": "string (required)",
  "password": "string (required, min 6 chars)"
}
```

Response (201):

```
{
  "message": "Admin registered successfully",
  "user": {
    "id": 1,
    "name": "Admin User",
    "email": "admin@example.com",
    "role": "ADMIN"
  }
}
```

POST /api/v1/auth/admin/signin

Admin login.

Request Body:

```
{
  "email": "string (required)",
  "password": "string (required)"
}
```

Response (200):

```
{
  "message": "Admin login successful",
  "user": {
    "id": 1,
    "name": "Admin User",
    "email": "admin@example.com",
    "role": "ADMIN"
  },
  "tokens": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
    "expiresIn": 3600
  }
}
```


PATCH /api/v1/auth/admin/:id/restore

Restore soft-deleted user (Admin only).

Response (200):


```
{
  "message": "User restored successfully"
}
```

User Management

GET /api/v1/user/me

Get current user profile (requires authentication).

Response (200):

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+1234567890",
  "address": "123 Main St",
  "role": "USER",
  "isActive": true,
  "profilePicture": "uploads/profile-123.jpg",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```


PUT /api/v1/user/profile-details

Update user profile details (requires authentication).

Request Body:


```
{
  "name": "string (optional)",
  "phone": "string (optional)",
  "address": "string (optional)"
}
```


Response (200):


```
{
  "message": "Profile updated successfully",
  "user": {
    "id": 1,
    "name": "Updated Name",
    "email": "john@example.com",
    "phone": "+1234567890",
    "address": "Updated Address"
  }
}
```


PUT /api/v1/user/profile-picture

Update user profile picture (requires authentication).

Request Body: FormData with file

```
Content-Type: multipart/form-data
image: <file>
```

Response (200):

```
{
  "message": "Profile picture updated successfully",
  "profilePicture": "uploads/profile-123.jpg"
}
```

GET /api/v1/user/admin/users

Get all users (Admin only).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `search`: string (optional)

Response (200):

```
{
  "users": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "role": "USER",
      "isActive": true,
      "createdAt": "2025-11-08T10:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 25,
    "pages": 3
  }
}
```

PATCH /api/v1/user/admin/user/:id/role

Update user role (Admin only).

Request Body:

```
{
  "role": "USER | ADMIN | STAFF"
}
```

Response (200):

```
{
  "message": "User role updated successfully"
}
```

DELETE /api/v1/user/admin/delete/user/:id

Delete user (Admin only).

Response (200):

```
{
  "message": "User deleted successfully"
}
```

Security Settings

POST /api/v1/security/change-password

Change password (requires authentication).

Request Body:

```
{
  "oldPassword": "string (required)",
  "newPassword": "string (required, min 6 chars)"
}
```

Response (200):

```
{
  "message": "Password changed successfully"
}
```

POST /api/v1/security/create-pin

Create PIN for additional security (requires authentication).

Request Body:

```
{
  "pin": "string (required, 4-6 digits)"
}
```

Response (200):

```
{
  "message": "PIN created successfully"
}
```

PATCH /api/v1/security/change-pin

Change PIN (requires authentication).

Request Body:

```
{
  "oldPin": "string (required)",
  "newPin": "string (required, 4-6 digits)"
}
```

Response (200):

```
{
  "message": "PIN changed successfully"
}
```

PATCH /api/v1/security/biometrics

Enable/disable biometric authentication (requires authentication).

Request Body:

```
{
  "enabled": true
}
```

Response (200):

```
{
  "message": "Biometric authentication enabled"
}
```

Services Management

POST /api/v1/services

Create new service (requires authentication).

Request Body:

```
{
  "type": "string (required)",
  "description": "string (required)",
  "price": "number (required, positive)",
  "estimatedDuration": "number (optional, minutes)"
}
```

Response (201):

```
{
  "id": 1,
  "type": "Wash & Fold",
  "description": "Complete laundry service",
  "price": 15.99,
  "estimatedDuration": 120,
  "isActive": true,
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```

GET /api/v1/services

Get all services (requires authentication).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `active`: boolean (optional)

Response (200):

```
{
  "services": [
    {
      "id": 1,
      "type": "Wash & Fold",
      "description": "Complete laundry service",
      "price": 15.99,
      "estimatedDuration": 120,
      "isActive": true,
      "createdAt": "2025-11-08T10:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 5,
    "pages": 1
  }
}
```


GET /api/v1/services/:id

Get service by ID (requires authentication).

Response (200):


```
{
  "id": 1,
  "type": "Wash & Fold",
  "description": "Complete laundry service",
  "price": 15.99,
  "estimatedDuration": 120,
  "isActive": true,
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```


PUT /api/v1/services/:id

Update service (requires authentication).

Request Body:

```
{
  "type": "string (optional)",
  "description": "string (optional)",
  "price": "number (optional)",
  "estimatedDuration": "number (optional)",
  "isActive": "boolean (optional)"
}
```

Response (200):

```
{
  "message": "Service updated successfully",
  "service": {
    "id": 1,
    "type": "Updated Service",
    "description": "Updated description",
    "price": 20.99,
    "estimatedDuration": 150,
    "isActive": true
  }
}
```

DELETE /api/v1/services/:id

Delete service (requires authentication).

Response (200):

```
{
  "message": "Service deleted successfully"
}
```

Bookings Management

POST /api/v1/bookings/create

Create new booking (requires authentication).

Request Body:

```
{
  "userId": "number (required)",
  "serviceId": "number (required)",
  "pickupAddress": "string (required)",
  "deliveryAddress": "string (required)",
  "date": "string (required, ISO date)",
  "totalAmount": "number (required, positive)",
  "deliveryFee": "number (optional, default 0)",
  "note": "string (optional)"
}
```

Response (201):

```
{
  "id": 1,
  "userId": 1,
  "serviceId": 1,
  "pickupAddress": "123 Main St",
  "deliveryAddress": "123 Main St",
  "date": "2025-11-10T10:00:00.000Z",
  "status": "PENDING",
  "totalAmount": 25.99,
  "deliveryFee": 5.0,
  "note": "Handle with care",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```

GET /api/v1/bookings/getAll

Get all bookings (requires authentication).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `status`: string (optional)
- `userId`: number (optional)
- `startDate`: string (optional)
- `endDate`: string (optional)

Response (200):

```
{
  "bookings": [
    {
      "id": 1,
      "userId": 1,
      "serviceId": 1,
      "pickupAddress": "123 Main St",
      "deliveryAddress": "123 Main St",
      "date": "2025-11-10T10:00:00.000Z",
      "status": "PENDING",
      "totalAmount": 25.99,
      "deliveryFee": 5.0,
      "createdAt": "2025-11-08T10:30:00.000Z",
      "user": {
        "id": 1,
        "name": "John Doe",
        "email": "john@example.com"
      },
      "service": {
        "id": 1,
        "type": "Wash & Fold",
        "price": 15.99
      }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 15,
    "pages": 2
  }
}
```


GET /api/v1/bookings/get/:id

Get booking by ID (requires authentication).

Response (200):

```
{
  "id": 1,
  "userId": 1,
  "serviceId": 1,
  "pickupAddress": "123 Main St",
  "deliveryAddress": "123 Main St",
  "date": "2025-11-10T10:00:00.000Z",
  "status": "PENDING",
  "totalAmount": 25.99,
  "deliveryFee": 5.0,
  "createdAt": "2025-11-08T10:30:00.000Z",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  },
  "service": {
    "id": 1,
    "type": "Wash & Fold",
    "price": 15.99
  }
}
```


PUT /api/v1/bookings/update/:id

Update booking (requires authentication).

Request Body:


```
{
  "pickupAddress": "string (optional)",
  "deliveryAddress": "string (optional)",
  "date": "string (optional)",
  "status": "PENDING | CONFIRMED | COMPLETED | CANCELLED (optional)",
  "totalAmount": "number (optional)",
  "deliveryFee": "number (optional)",
  "note": "string (optional)"
}
```


Response (200):

```
{
  "message": "Booking updated successfully",
  "booking": {
    "id": 1,
    "status": "CONFIRMED",
    "pickupAddress": "Updated Address"
  }
}
```

DELETE /api/v1/bookings/delete/:id

Delete booking (requires authentication).

Response (200):

```
{
  "message": "Booking deleted successfully"
}
```

Payments Management

POST /api/v1/payments/create

Create new payment (requires authentication).

Request Body:

```
{
  "bookingId": "number (required)",
  "amount": "number (required, positive)",
  "method": "CASH | CARD | ONLINE (required)",
  "status": "PENDING | COMPLETED | FAILED (optional, default PENDING)",
  "transactionId": "string (optional)"
}
```

Response (201):

```
{
  "id": 1,
  "bookingId": 1,
  "amount": 25.99,
  "method": "CARD",
  "status": "COMPLETED",
  "transactionId": "txn_123456789",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```


GET /api/v1/payments/getAll

Get all payments (requires authentication).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `status`: string (optional)
- `method`: string (optional)
- `startDate`: string (optional)
- `endDate`: string (optional)

Response (200):

```
{
  "payments": [
    {
      "id": 1,
      "bookingId": 1,
      "amount": 25.99,
      "method": "CARD",
      "status": "COMPLETED",
      "transactionId": "txn_123456789",
      "createdAt": "2025-11-08T10:30:00.000Z",
      "booking": {
        "id": 1,
        "user": {
          "id": 1,
          "name": "John Doe"
        },
        "service": {
          "id": 1,
          "type": "Wash & Fold"
        }
      }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 20,
    "pages": 2
  }
}
```


GET /api/v1/payments/get/:id

Get payment by ID (requires authentication).

Response (200):

```
{
  "id": 1,
  "bookingId": 1,
  "amount": 25.99,
  "method": "CARD",
  "status": "COMPLETED",
  "transactionId": "txn_123456789",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```


PUT /api/v1/payments/update/:id

Update payment (requires authentication).

Request Body:


```
{
  "amount": "number (optional)",
  "method": "string (optional)",
  "status": "string (optional)",
  "transactionId": "string (optional)"
}
```


Response (200):


```
{
  "message": "Payment updated successfully",
  "payment": {
    "id": 1,
    "status": "COMPLETED",
    "transactionId": "txn_updated"
  }
}
```


DELETE /api/v1/payments/delete/:id

Delete payment (requires authentication).

Response (200):


```
{
  "message": "Payment deleted successfully"
}
```

Invoices Management

POST /api/v1/invoices/create

Create new invoice (Staff/Admin only).

Request Body:

```
{
  "paymentId": "number (required)",
  "invoiceNo": "string (optional, auto-generated)",
  "totalAmount": "number (required)",
  "tax": "number (optional, default 0)",
  "discount": "number (optional, default 0)",
  "dueDate": "string (required, ISO date)",
  "notes": "string (optional)"
}
```

Response (201):

```
{
  "id": 1,
  "paymentId": 1,
  "invoiceNo": "INV-2025-001",
  "totalAmount": 25.99,
  "tax": 2.6,
  "discount": 0,
  "dueDate": "2025-11-15T00:00:00.000Z",
  "status": "UNPAID",
  "issuedAt": "2025-11-08T10:30:00.000Z",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```

GET /api/v1/invoices/getAll

Get all invoices (requires authentication).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `status`: string (optional)
- `startDate`: string (optional)
- `endDate`: string (optional)

- `search`: string (optional)

Response (200):


```
{
  "invoices": [
    {
      "id": 1,
      "invoiceNo": "INV-2025-001",
      "totalAmount": 25.99,
      "tax": 2.6,
      "discount": 0,
      "status": "UNPAID",
      "issuedAt": "2025-11-08T10:30:00.000Z",
      "dueDate": "2025-11-15T00:00:00.000Z",
      "payment": {
        "id": 1,
        "booking": {
          "user": {
            "id": 1,
            "name": "John Doe",
            "email": "john@example.com"
          },
          "service": {
            "id": 1,
            "type": "Wash & Fold"
          }
        }
      }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 12,
    "pages": 2
  }
}
```


GET /api/v1/invoices/get/:id

Get invoice by ID (requires authentication).

Response (200):

```
{
  "id": 1,
  "invoiceNo": "INV-2025-001",
  "totalAmount": 25.99,
  "tax": 2.6,
  "discount": 0,
  "status": "UNPAID",
  "issuedAt": "2025-11-08T10:30:00.000Z",
  "dueDate": "2025-11-15T00:00:00.000Z"
}
```


PUT /api/v1/invoices/update/:id

Update invoice (Staff/Admin only).

Request Body:


```
{
  "totalAmount": "number (optional)",
  "tax": "number (optional)",
  "discount": "number (optional)",
  "dueDate": "string (optional)",
  "status": "string (optional)",
  "notes": "string (optional)"
}
```


Response (200):


```
{
  "message": "Invoice updated successfully",
  "invoice": {
    "id": 1,
    "status": "PAID",
    "totalAmount": 28.59
  }
}
```


DELETE /api/v1/invoices/delete/:id

Delete invoice (Staff/Admin only).

Response (200):

```
{
  "message": "Invoice deleted successfully"
}
```

PATCH /api/v1/invoices/markPaid/:id

Mark invoice as paid (Staff/Admin only).

Response (200):

```
{
  "message": "Invoice marked as paid",
  "invoice": {
    "id": 1,
    "status": "PAID"
  }
}
```

GET /api/v1/invoices/pdf/:id

Generate and download invoice PDF (requires authentication).

Response: PDF file download

GET /api/v1/invoices/report

Generate invoices report (Staff/Admin only).

Query Parameters:

- `format`: 'excel' | 'json' (default: json)
- `status`: string (optional)
- `startDate`: string (optional)
- `endDate`: string (optional)

Response (200) - JSON:

```
{
  "data": [...invoices],
  "format": "json"
}
```

Response (200) - Excel: Excel file download

GET /api/v1/invoices/stats

Get invoice statistics (Staff/Admin only).

Response (200):

```
{
  "totalInvoices": 25,
  "paidInvoices": 20,
  "unpaidInvoices": 5,
  "overdueInvoices": 2,
  "totalRevenue": 1250.50,
  "unpaidAmount": 125.75,
  "monthlyRevenue": [...]
}
```

GET /api/v1/invoices/overdue

Get overdue invoices (Staff/Admin only).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)

Response (200):

```
{
  "invoices": [...overdueInvoices],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 5,
    "pages": 1
  }
}
```

Receipts Management

POST /api/v1/receipts/create

Create new receipt (Staff/Admin only).

Request Body:

```
{
  "invoiceId": "number (required)",
  "receiptNo": "string (optional, auto-generated)",
  "receivedBy": "string (optional)",
  "notes": "string (optional)"
}
```

Response (201):

```
{
  "id": 1,
  "invoiceId": 1,
  "receiptNo": "RCP-2025-001",
  "receivedBy": "John Smith",
  "issuedAt": "2025-11-08T10:30:00.000Z",
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```

GET /api/v1/receipts/getAll

Get all receipts (requires authentication).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `startDate`: string (optional)
- `endDate`: string (optional)
- `search`: string (optional)

Response (200):

```
{
  "receipts": [
    {
      "id": 1,
      "receiptNo": "RCP-2025-001",
      "receivedBy": "John Smith",
      "issuedAt": "2025-11-08T10:30:00.000Z",
      "invoice": {
        "id": 1,
        "invoiceNo": "INV-2025-001",
        "payment": {
          "amount": 25.99,
          "booking": {
            "user": {
              "name": "John Doe"
            },
            "service": {
              "type": "Wash & Fold"
            }
          }
        }
      }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 8,
    "pages": 1
  }
}
```


GET /api/v1/receipts/get/:id

Get receipt by ID (requires authentication).

Response (200):


```
{
  "id": 1,
  "receiptNo": "RCP-2025-001",
  "receivedBy": "John Smith",
  "issuedAt": "2025-11-08T10:30:00.000Z"
}
```

PUT /api/v1/receipts/update/:id

Update receipt (Staff/Admin only).

Request Body:

```
{
  "receivedBy": "string (optional)",
  "notes": "string (optional)"
}
```

Response (200):

```
{
  "message": "Receipt updated successfully",
  "receipt": {
    "id": 1,
    "receivedBy": "Updated Name"
  }
}
```

DELETE /api/v1/receipts/delete/:id

Delete receipt (Staff/Admin only).

Response (200):

```
{
  "message": "Receipt deleted successfully"
}
```

GET /api/v1/receipts/pdf/:id

Generate and download receipt PDF (requires authentication).

Response: PDF file download

GET /api/v1/receipts/report

Generate receipts report (Staff/Admin only).

Query Parameters:

- `format`: 'excel' | 'json' (default: json)
- `startDate`: string (optional)

- `endDate`: string (optional)

Response (200) - JSON:

```
{
  "data": [...receipts],
  "format": "json"
}
```

Response (200) - Excel: Excel file download

GET /api/v1/receipts/stats

Get receipt statistics (Staff/Admin only).

Response (200):

```
{
  "totalReceipts": 15,
  "totalAmount": 750.25,
  "averageReceipt": 50.02,
  "monthlyStats": [...]
}
```

Testimonials Management

GET /api/v1/testimonials

Get all approved testimonials (public endpoint).

Query Parameters:

- `page`: number (default: 1)
- `limit`: number (default: 10)
- `rating`: number (optional, 1-5)

Response (200):

```
{
  "testimonials": [
    {
      "id": 1,
      "userId": 1,
      "rating": 5,
      "comment": "Excellent service!",
      "isApproved": true,
      "createdAt": "2025-11-08T10:30:00.000Z",
      "user": {
        "id": 1,
        "name": "John Doe"
      }
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 12,
    "pages": 2
  }
}
```

POST /api/v1/testimonials

Create new testimonial (requires authentication).

Request Body:

```
{
  "rating": "number (required, 1-5)",
  "comment": "string (required, max 500 chars)",
  "serviceId": "number (optional)"
}
```

Response (201):

```
{
  "id": 1,
  "userId": 1,
  "rating": 5,
  "comment": "Excellent service!",
  "isApproved": false,
  "createdAt": "2025-11-08T10:30:00.000Z"
}
```


GET /api/v1/testimonials/user/me

Get current user's testimonials (requires authentication).

Response (200):


```
{
  "testimonials": [
    {
      "id": 1,
      "rating": 5,
      "comment": "Excellent service!",
      "isApproved": false,
      "createdAt": "2025-11-08T10:30:00.000Z"
    }
  ]
}
```


GET /api/v1/testimonials/:id

Get testimonial by ID (requires authentication).

Response (200):

```
{
  "id": 1,
  "userId": 1,
  "rating": 5,
  "comment": "Excellent service!",
  "isApproved": false,
  "createdAt": "2025-11-08T10:30:00.000Z",
  "user": {
    "id": 1,
    "name": "John Doe"
  }
}
```

PUT /api/v1/testimonials/:id

Update testimonial (requires authentication, owner only).

Request Body:

```
{
  "rating": "number (optional, 1-5)",
  "comment": "string (optional, max 500 chars)"
}
```

Response (200):

```
{
  "message": "Testimonial updated successfully",
  "testimonial": {
    "id": 1,
    "rating": 4,
    "comment": "Updated comment"
  }
}
```

DELETE /api/v1/testimonials/:id

Delete testimonial (requires authentication, owner only).

Response (200):

```
{
  "message": "Testimonial deleted successfully"
}
```

PUT /api/v1/testimonials/:id/approve

Approve testimonial (Admin only).

Response (200):

```
{
  "message": "Testimonial approved successfully"
}
```

GET /api/v1/testimonials/admin/stats

Get testimonial statistics (Admin only).

Response (200):

```
{
  "totalTestimonials": 25,
  "approvedTestimonials": 20,
  "pendingTestimonials": 5,
  "averageRating": 4.2,
  "ratingDistribution": {
    "1": 1,
    "2": 0,
    "3": 2,
    "4": 5,
    "5": 17
  }
}
```

Reports & Analytics

GET /api/v1/reports/dashboard

Get dashboard statistics (Staff/Admin only).

Response (200):

```
{
  "totalCustomers": 150,
  "totalBookings": 320,
  "monthlyRevenue": 2500.50,
  "yearlyRevenue": 28500.75,
  "pendingBookings": 12,
  "completedBookings": 308,
  "unpaidInvoices": 8,
  "recentPayments": [...]
}
```

GET /api/v1/reports/monthly-trends

Get monthly revenue and booking trends (Staff/Admin only).

Response (200):

```
[
  {
    "month": "Nov 2025",
    "revenue": 2500.5,
    "bookings": 45
  },
  {
    "month": "Oct 2025",
    "revenue": 2200.25,
    "bookings": 38
  }
]
```

GET /api/v1/reports/financial

Get financial report (Staff/Admin only).

Query Parameters:

- `startDate`: string (required)
- `endDate`: string (required)
- `format`: 'excel' | 'json' (default: json)

Response (200) - JSON:

```
{
  "summary": {
    "totalRevenue": 2500.50,
    "totalPayments": 45,
    "averageOrderValue": 55.57,
    "paidInvoices": 42,
    "unpaidInvoices": 3,
    "totalTax": 250.05,
    "totalDiscounts": 25.00
  },
  "payments": [...],
  "revenueByService": [...]
}
```

Response (200) - Excel: Excel file download

GET /api/v1/reports/customers

Get customer analytics report (Staff/Admin only).

Query Parameters:

- `startDate`: string (optional)
- `endDate`: string (optional)
- `status`: 'active' | 'inactive' (optional)
- `format`: 'excel' | 'json' (default: json)

Response (200) - JSON:

```
{
  "customers": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "totalBookings": 5,
      "totalSpent": 125.5,
      "lastBooking": "2025-11-05T10:00:00.000Z",
      "status": "Active"
    }
  ]
}
```

Response (200) - Excel: Excel file download

GET /api/v1/reports/services

Get service performance report (Staff/Admin only).

Query Parameters:

- `startDate`: string (required)
- `endDate`: string (required)
- `format`: 'excel' | 'json' (default: json)

Response (200) - JSON:

```
{
  "services": [
    {
      "serviceName": "Wash & Fold",
      "totalBookings": 25,
      "totalRevenue": 399.75,
      "averageRevenue": 15.99,
      "completedBookings": 23,
      "cancelledBookings": 2,
      "conversionRate": 92
    }
  ]
}
```

Response (200) - Excel: Excel file download

POST /api/v1/reports/generate

Manually trigger scheduled report generation (Staff/Admin only).

Response (200):

```
{
  "message": "Reports generated successfully"
}
```

GET /api/v1/reports/cron-status

Get cron job status (Staff/Admin only).

Response (200):

```
{
  "jobs": [
    {
      "name": "daily-reports",
      "status": "running",
      "nextRun": "2025-11-09T00:00:00.000Z",
      "lastRun": "2025-11-08T00:00:00.000Z"
    }
  ]
}
```

Admin Dashboard

GET /api/v1/admin/dashboard

Get admin dashboard data (Admin only).

Response (200):


```
{
  "totalUsers": 150,
  "totalBookings": 320,
  "totalRevenue": 28500.75,
  "activeUsers": 120,
  "pendingBookings": 12,
  "monthlyGrowth": 15.5,
  "recentActivity": [...]
}
```


GET /api/v1/admin/reports/bookings

Get detailed booking reports (Admin only).

Query Parameters:

- `startDate`: string (optional)
- `endDate`: string (optional)
- `status`: string (optional)

Response (200):

```
{
  "bookings": [...],
  "summary": {
    "totalBookings": 320,
    "completedBookings": 308,
    "cancelledBookings": 12,
    "revenue": 28500.75
  }
}
```

Error Responses

All endpoints may return the following error responses:

400 Bad Request

```
{
  "error": "Validation failed",
  "details": ["Field is required", "Invalid email format"]
}
```

401 Unauthorized

```
{
  "error": "Unauthorized",
  "message": "Invalid or expired token"
}
```

403 Forbidden

```
{
  "error": "Forbidden",
  "message": "Insufficient permissions"
}
```

404 Not Found

```
{
  "error": "Not Found",
  "message": "Resource not found"
}
```

429 Too Many Requests

```
{
  "error": "Too many requests",
  "message": "Rate limit exceeded. Try again later."
}
```

500 Internal Server Error

```
{
  "error": "Internal Server Error",
  "message": "Something went wrong"
}
```

File Upload

The API supports file uploads for profile pictures. Files are stored in the `uploads/` directory and served statically.

Supported formats: JPEG, PNG, GIF **Maximum file size:** 10MB **Upload endpoint:** `PUT /api/v1/user/profile-picture`

Data Models

User

```
{
  id: number;
  name: string;
  email: string;
  phone?: string;
  address?: string;
  role: 'USER' | 'ADMIN' | 'STAFF';
  isActive: boolean;
  profilePicture?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Service

```
{
  id: number;
  type: string;
  description: string;
  price: number;
  estimatedDuration?: number;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}
```

Booking

```
{
  id: number;
  userId: number;
  serviceId: number;
  pickupAddress: string;
  deliveryAddress: string;
  date: Date;
  status: 'PENDING' | 'CONFIRMED' | 'COMPLETED' | 'CANCELLED';
  totalAmount: number;
  deliveryFee: number;
  note?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Payment

```
{
  id: number;
  bookingId: number;
  amount: number;
  method: 'CASH' | 'CARD' | 'ONLINE';
  status: 'PENDING' | 'COMPLETED' | 'FAILED';
  transactionId?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Invoice

```
{
  id: number;
  paymentId: number;
  invoiceNo: string;
  totalAmount: number;
  tax: number;
  discount: number;
  status: 'PAID' | 'UNPAID';
  issuedAt: Date;
  dueDate: Date;
  notes?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Receipt

```
{
  id: number;
  invoiceId: number;
  receiptNo: string;
  receivedBy?: string;
  issuedAt: Date;
  notes?: string;
  createdAt: Date;
  updatedAt: Date;
}
```

Testimonial

```
{
  id: number;
  userId: number;
  serviceId?: number;
  rating: number; // 1-5
  comment: string;
  isApproved: boolean;
  createdAt: Date;
  updatedAt: Date;
}
```

Rate Limiting

- **Global limit:** 100 requests per 15 minutes per IP
- **Authentication endpoints:** Additional limits may apply
- Rate limit headers are included in responses

Security Features

- **JWT Authentication** with refresh tokens
- **Password hashing** with bcrypt
- **Input validation** with Zod schemas
- **XSS protection** and sanitization
- **CORS configuration**
- **Helmet security headers**
- **Rate limiting**
- **File upload validation**

Development

Environment Variables:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://...
JWT_SECRET=your-secret-key
JWT_REFRESH_SECRET=your-refresh-secret
FRONTEND_URL=http://localhost:3001
ENABLE_CRON_JOBS=false
```

Available Scripts:

- `yarn dev` - Development server with hot reload
- `yarn build` - Production build
- `yarn start` - Production server
- `yarn lint` - ESLint checking
- `yarn lint:fix` - Auto-fix ESLint issues

This documentation provides comprehensive coverage of all API endpoints, request/response formats, authentication requirements, and data models for Tyler's Laundry API.